

Agentic Restaurant Customer Service System

*A Multi-Agent Natural Language Processing System
Built with LangGraph, LangChain, and GPT-4o-mini*

Course: CSE389 (UG2023) — Natural Language Processing

Students: Mohamed Tarek Elsayed ID: 2300535
Renad Sameh Ibrahim ID: 23p0292
Salma Tamer Fathi ID: 23p0023

Supervisor: Prof. Alaa Hmady

TA: Nada Amr

Date: May 2026

Abstract

This report presents the design, implementation, and evaluation of a multi-agent customer service chatbot for a restaurant. The system routes every customer message through an intent-classification orchestrator to one of four specialised sub-agents — Menu, Order, FAQ, and Support — each modelled as a directed acyclic graph of LLM-powered and deterministic nodes using LangGraph. A FastAPI backend, a SQLAlchemy database layer, and a React front-end complete the production stack. The entire pipeline is evaluated using an LLM-as-a-Judge framework that scores responses on relevance, correctness, and helpfulness across 50 representative test cases.

Contents

1	Introduction	5
1.1	Motivation	5
1.2	Problem Statement	5
1.3	Objectives	5
1.4	Report Structure	6
2	System Architecture	7
2.1	High-Level Design	7
2.2	Shared State	7
2.3	Session Persistence	8
3	Technology Stack	9
4	Orchestrator — Intent Classifier	10
4.1	Responsibility	10
4.2	Graph Structure	10
4.3	Nodes	11
4.4	Routing Logic	11
5	Menu Agent	12
5.1	Responsibility	12
5.2	Graph Structure	12
5.3	Nodes	13
5.4	Menu Tools	13
5.5	Reflection Loop	13
6	FAQ Agent	15
6.1	Responsibility	15
6.2	Graph Structure	15
6.3	Nodes	16
6.4	Retrieval Strategy	16
7	Order Agent	17
7.1	Responsibility	17
7.2	Graph Structure	17
7.3	Nodes	18
7.4	Context-Aware Routing	18
7.5	Multi-Turn Modification Flow	19

8	Support Agent	20
8.1	Responsibility	20
8.2	Graph Structure	20
8.3	Nodes	21
8.4	Response Tone	21
9	Backend API	23
9.1	FastAPI Architecture	23
9.2	Chat Endpoint Flow	23
9.3	Database Layer	23
10	Evaluation	25
10.1	Methodology — LLM-as-a-Judge	25
10.2	Test Suite	25
10.3	Results	26
10.3.1	Per-Agent Scores by Dimension	26
10.3.2	Pass Rate and Overall Average	27
10.3.3	Score Heatmap	28
10.3.4	Radar Chart — Dimension Profile	29
10.4	Intent Classification Accuracy	29
11	Conclusion	30
11.1	Summary	30
11.2	Limitations	30
11.3	Future Work	31
A	Project Structure	32

List of Figures

4.1	Orchestrator Agent Graph	10
5.1	Menu Agent Graph	12
5.2	Menu Agent Reflection Loop	14
6.1	FAQ Agent Graph	15
7.1	Order Agent Graph	17
8.1	Support Agent Graph	20
10.1	Average Relevance, Correctness, and Helpfulness per agent (1–5 scale). The dashed line marks the pass threshold of 3.0.	26
10.2	Pass rate (bars, left axis) and overall average score (line, right axis) per agent.	27
10.3	Per-test score heatmap. Green = high score (4–5), red = low score (1–2).	28
10.4	Radar chart comparing Relevance, Correctness, and Helpfulness across all agents.	29

List of Tables

2.1	Key <code>MainState</code> Fields	8
3.1	Technology Stack Overview	9
4.1	Orchestrator Node Descriptions	11
5.1	Menu Agent Node Descriptions	13
5.2	Menu Agent Tools	13
6.1	FAQ Agent Node Descriptions	16
7.1	Order Agent Node Descriptions	18
8.1	Support Agent Node Descriptions	21
9.1	API Endpoints	23
10.1	Evaluation Dimensions	25
10.2	Test Suite Composition	26
10.3	Intent Classification Accuracy (8 test cases)	29

Chapter 1

Introduction

1.1 Motivation

The restaurant industry increasingly relies on digital channels to handle customer inquiries, process orders, and resolve complaints at scale. Traditional rule-based chatbots struggle with the open-ended, ambiguous nature of natural language. This project applies modern large language model (LLM) techniques to build an *agentic* customer service assistant that can reason, plan, and act — not merely pattern-match.

1.2 Problem Statement

A single monolithic LLM prompt cannot handle the full breadth of restaurant customer interactions efficiently. Different types of requests (browsing the menu, placing an order, asking about opening hours, filing a complaint) require different workflows, tools, and database operations. The challenge is to correctly identify the user’s intent and delegate it to a sub-system optimised for that task — all while maintaining conversational context across multiple turns.

1.3 Objectives

- Design a modular, multi-agent architecture where each agent is responsible for exactly one domain of customer interaction.
- Use LangGraph StateGraph to make agent logic explicit, inspectable, and testable as directed graphs of nodes and edges.
- Ground every agent in real database state (menu items, orders, FAQs, tickets) via SQLAlchemy repositories.
- Expose the system through a RESTful FastAPI backend consumed by a React single-page application.
- Evaluate quality objectively using an LLM-as-a-Judge scoring framework.

1.4 Report Structure

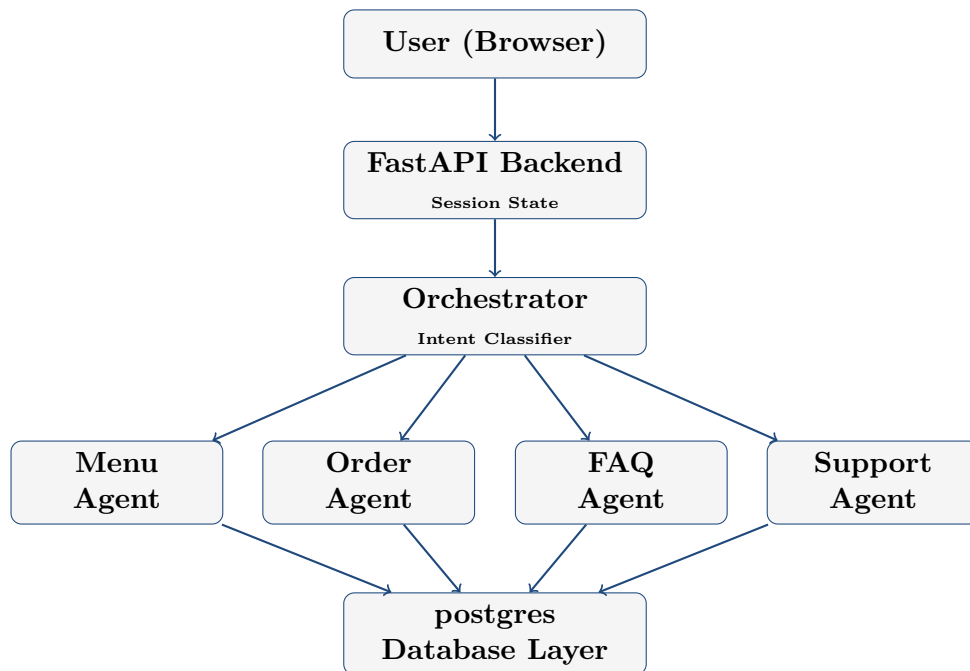
Chapter 2 describes the overall system architecture. Chapter 3 surveys the technology stack. Chapters 4 through 8 detail each agent individually. Chapter 9 covers the FastAPI backend and session management. Chapter 10 presents the evaluation methodology and results. Chapter 11 concludes with lessons learned and future directions.

Chapter 2

System Architecture

2.1 High-Level Design

The system follows a **hierarchical multi-agent** pattern:



2.2 Shared State

All agents communicate through a single **TypedDict** called **MainState**. LangGraph uses this as the canonical state object that is threaded through every node invocation. Key fields include:

Table 2.1: Key MainState Fields

Field	Purpose
<code>user_message</code>	Raw text from the customer
<code>intent</code>	Classified intent (<i>menu / order / faq / support / off_topic / unclear</i>)
<code>response</code>	Final text response returned to the user
<code>messages</code>	Full conversation history (LangChain message objects)
<code>customer_id</code>	Authenticated customer primary key
<code>extracted_order</code>	Structured order payload (items, type, address)
<code>extracted_complaint</code>	Structured complaint payload
<code>order_awaiting_confirmation</code>	Flag: pending user confirmation before placing
<code>modification_target_id</code>	Order ID being modified across turns
<code>needs_human</code>	Flag: complaint requires human escalation
<code>faq</code>	Retrieved FAQ object from the database
<code>tool_result</code>	Raw tool output from the last executed tool
<code>clarification_attempts</code>	Counter to prevent infinite clarification loops

2.3 Session Persistence

The FastAPI endpoint maintains an in-memory `_session_agent_state` dictionary keyed by `session_id`. Between API calls, a curated set of fields called `PERSISTENT_FIELDS` is preserved so the agent retains context across turns:

```

1 PERSISTENT_FIELDS = (
2     "extracted_order", "order_confirmed",
3     "order_awaiting_confirmation",
4     "extracted_complaint", "needs_human",
5     "customer_id",
6     "modification_target_id",
7 )

```

Listing 2.1: Persistent fields in the chat endpoint

Chapter 3

Technology Stack

Table 3.1: Technology Stack Overview

Layer	Technology	Role
LLM	OpenAI GPT-4o-mini	Reasoning, extraction, generation for all nodes
Agent Framework	LangGraph 0.x	Directed graph execution engine for all sub-agents
LLM Toolkit	LangChain	Prompt templates, message types, tool binding
Structured Output	Pydantic v2	Schema validation for all LLM responses
Web Framework	FastAPI	REST API, dependency injection, auth middleware
ORM	SQLAlchemy	Database models and repositories
Database	PostgreSQL / SQLite	Persistent storage for orders, menu, FAQs, tickets
Migrations	Alembic	Schema versioning
Frontend	React + Vite	Single-page application
Embeddings	OpenAI Embeddings	Semantic FAQ retrieval and menu search

Why LangGraph?

LangGraph models each agent as an explicit directed graph with typed state. This gives three concrete advantages over simple LLM chains:

1. **Inspectability** — the graph can be rendered as a Mermaid diagram, making the control flow visible and debuggable.
2. **Deterministic routing** — critical transitions (e.g. always validate before placing an order) are hard-coded graph edges that the LLM cannot bypass.
3. **Iterative refinement** — nodes like *reflection* allow the agent to check its own output and loop back if unsatisfied.

Chapter 4

Orchestrator — Intent Classifier

4.1 Responsibility

The orchestrator is the entry point for every customer message. Its sole job is to classify the user's intent and delegate control to the appropriate sub-agent. It does not generate any content of its own (except for clarifying questions and off-topic replies).

4.2 Graph Structure

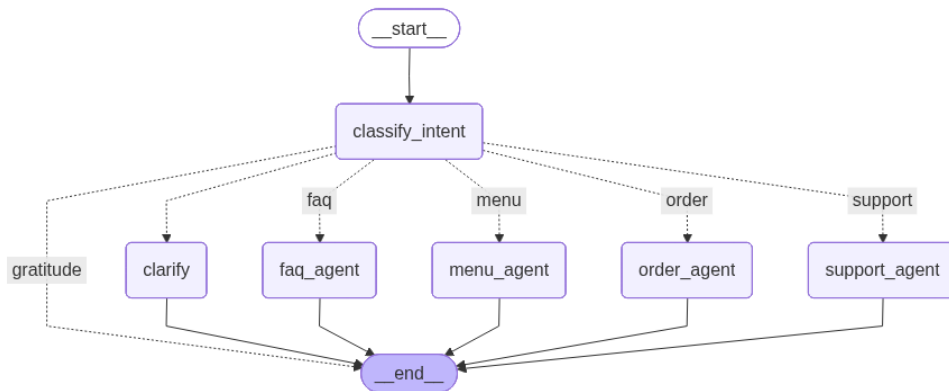


Figure 4.1: Orchestrator Agent Graph

4.3 Nodes

Table 4.1: Orchestrator Node Descriptions

Node	Description
<code>classify_intent</code>	LLM node. Classifies the user message into one of six intents: <i>menu</i> , <i>order</i> , <i>faq</i> , <i>support</i> , <i>off_topic</i> , or <i>unclear</i> . Uses structured output via Pydantic <code>IntentDecision</code> . Includes an exponential-backoff retry on API errors.
<code>clarify</code>	Deterministic node. Returns the LLM-generated clarifying question when intent is <i>unclear</i> . Increments <code>clarification_attempts</code> ; after two failures defaults to <i>faq</i> .
<code>menu_agent</code>	Sub-agent wrapper. Invokes <code>build_menu_graph()</code> and propagates the response back to orchestrator state.
<code>order_agent</code>	Sub-agent wrapper. Invokes <code>build_order_agent_graph()</code> and carries back order-related fields (<code>extracted_order</code> , <code>order_id</code> , etc.).
<code>faq_agent</code>	Sub-agent wrapper. Invokes <code>build_faq_graph()</code> and returns the response.
<code>support_agent</code>	Sub-agent wrapper. Invokes <code>build_support_agent_graph()</code> and returns the response.

4.4 Routing Logic

- **Gratitude short-circuit** — before calling the LLM, a token-based check detects thank-you messages and generates a context-aware closing reply without classification overhead.
- **Off-topic guard** — the LLM is instructed to classify weather, sports, coding, or other non-restaurant topics as *off_topic*, which triggers a polite redirection.
- **Clarification loop** — *unclear* intent causes a clarifying question; after two rounds the system defaults to *faq* to avoid frustrating the user.

Chapter 5

Menu Agent

5.1 Responsibility

The Menu Agent answers all questions about what the restaurant offers: full menu listings, item-specific price queries, dietary-preference recommendations, and semantic food suggestions (e.g. “something light and healthy”).

5.2 Graph Structure

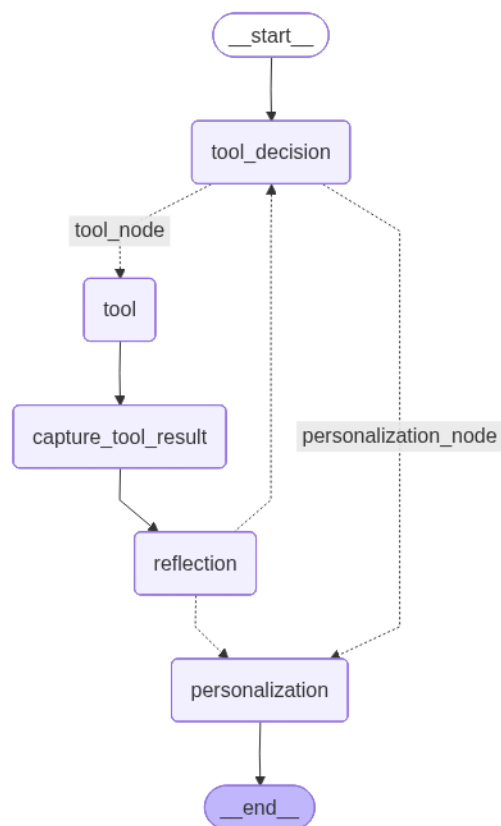


Figure 5.1: Menu Agent Graph

5.3 Nodes

Table 5.1: Menu Agent Node Descriptions

Node	Description
<code>tool_decision</code>	LLM node with tool-binding. Receives the user query and conversation history, selects and calls the most appropriate menu tool, and returns a draft response. The LLM is bound to four tools (see Section 5.4) via <code>llm.bind_tools()</code> .
<code>tool</code>	Prebuilt LangGraph <code>ToolNode</code> . Executes whichever tool the LLM selected and stores the raw result in state.
<code>capture_tool_result</code>	Deterministic node. Extracts the tool output from the LangChain message list and writes it to <code>state["tool_result"]</code> for subsequent nodes.
<code>reflection</code>	LLM node. Reviews the tool result and draft response against the original query. Returns a structured <code>ReflectionDecision</code> (<code>satisfied: bool</code>). Prevents hallucinated or irrelevant answers.
<code>personalization</code>	LLM node. Formats the final answer with warm tone, correct price formatting (\$XX.XX), and bulleted lists where appropriate. Returns <code>state["response"]</code> .

5.4 Menu Tools

Table 5.2: Menu Agent Tools

Tool	When Used
<code>get_all_menu_items</code>	User asks to see the full menu or “what do you have?”
<code>get_menu_item_by_name</code>	User asks about a specific named item (e.g. “how much is the Burger?”)
<code>search_menu_by_keyword</code>	A literal keyword appears in the query (e.g. “pizza”, “chicken”)
<code>search_menu_semantically</code>	User expresses a preference or mood (e.g. “something light”, “comfort food”) — uses OpenAI embeddings

5.5 Reflection Loop

The graph uses a conditional edge from `reflection` back to `tool_decision` if the reflection model is unsatisfied. This creates a self-correcting loop bounded by `max_iterations` (default 3), preventing infinite retries.

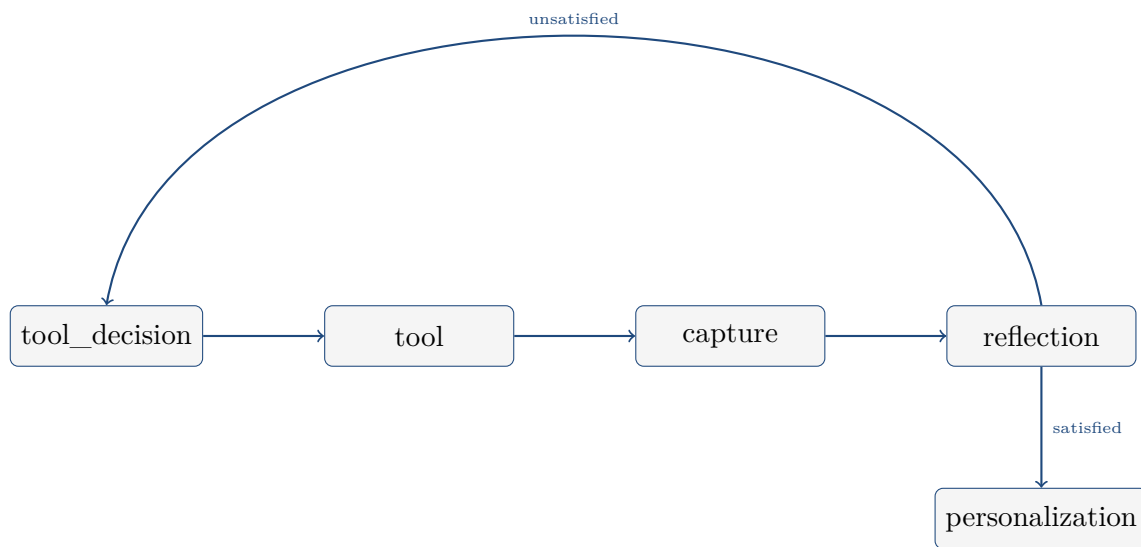


Figure 5.2: Menu Agent Reflection Loop

Chapter 6

FAQ Agent

6.1 Responsibility

The FAQ Agent handles general restaurant questions: operating hours, location, parking, reservation policy, delivery areas, and dietary accommodation. It retrieves the most semantically similar FAQ entry from the database and generates a natural-language answer.

6.2 Graph Structure

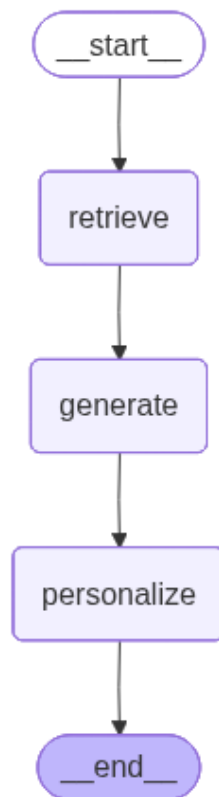


Figure 6.1: FAQ Agent Graph

6.3 Nodes

Table 6.1: FAQ Agent Node Descriptions

Node	Description
<code>retrieve</code>	Deterministic node. Calls <code>find_best_faq()</code> which computes cosine similarity between the user question embedding and all stored FAQ embeddings, returning the best match and its similarity score.
<code>generate</code>	LLM node. Given the matched FAQ answer and the last 10 turns of conversation history, generates a natural-language response tailored to the customer’s phrasing. Falls back gracefully if no FAQ was found.
<code>personalize</code>	Deterministic node. Appends a friendly emoji and prints debug state. Produces <code>state["response"]</code> .

6.4 Retrieval Strategy

FAQ retrieval uses embedding-based semantic similarity rather than keyword matching. Customer questions are embedded with the same OpenAI model used to embed FAQ entries at database-seed time. The FAQ with the highest cosine similarity is selected. This handles paraphrasing robustly (e.g. “Are you open Saturday?” matches “What are your weekend hours?”).

Chapter 7

Order Agent

7.1 Responsibility

The Order Agent is the most complex sub-agent. It handles the complete order lifecycle: new order extraction and validation, confirmation, placement, in-cart modification, cancellation, and post-placement order modification (changing order type, adding items to an already-placed order).

7.2 Graph Structure

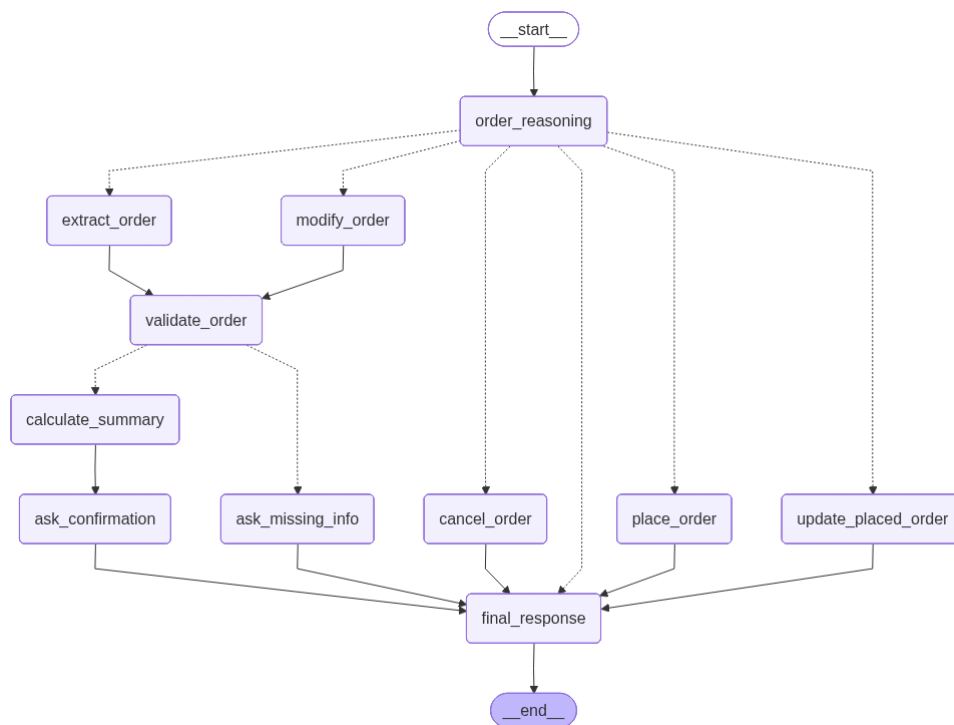


Figure 7.1: Order Agent Graph

7.3 Nodes

Table 7.1: Order Agent Node Descriptions

Node	Description
<code>order_reasoning</code>	Central LLM reasoning node. Receives full state (cart, history, intent, modification target) and decides the next step from a restricted set: <code>{extract_order, modify_order, place_order, cancel_order, update_placed_order, final_response}</code> . Uses <code>ReasoningDecision</code> Pydantic schema.
<code>extract_order</code>	LLM node. Extracts order fields (items, quantities, order type, delivery address) from the user message and merges them into <code>extracted_order</code> .
<code>validate_order</code>	Deterministic node. Calls <code>validate_order_items()</code> against the database to check that requested menu items exist. Sets <code>order_ready</code> and <code>missing_fields</code> .
<code>ask_missing_info</code>	LLM node. Generates a targeted question asking for whichever field is missing (order type, delivery address, unknown items).
<code>calculate_summary</code>	Deterministic node. Calls <code>calculate_order_total()</code> and builds a human-readable order summary.
<code>ask_confirmation</code>	Deterministic node. Presents the full order summary and asks the customer to confirm. Sets <code>order_awaiting_confirmation = True</code> .
<code>modify_order</code>	LLM node. Applies in-cart changes (quantity updates, item swaps) to <code>extracted_order</code> before re-validation.
<code>place_order</code>	Tool node. Calls <code>create_order()</code> and <code>create_order_items()</code> to write the order to the database. Creates a <code>Delivery</code> record if needed.
<code>cancel_order</code>	Tool node. Extracts the order ID via regex from the user message and calls <code>cancel_order()</code> on the repository.
<code>update_placed_order</code>	LLM + tool node. Modifies an already-placed order. Extracts a new order type or items to add from the user message and calls <code>update_placed_order()</code> . Persists <code>modification_target_id</code> for multi-turn flows.
<code>final_response</code>	Deterministic node. Returns the accumulated <code>state["response"]</code> to the orchestrator.

7.4 Context-Aware Routing

The `order_reasoning` node proposes a next step, but a post-processing function `_coerce_next_step()` applies deterministic overrides to prevent the LLM from making unsafe routing decisions:

- **Update coercion** — if `_wants_to_update_placed_order()` returns `True`, the step is forced to `update_placed_order` regardless of what the LLM proposed.
- **Cancel coercion** — if the last AI message asked for a cancel order ID and the user replied with a bare number, the step is coerced to `cancel_order`.

- **Empty cart guard** — if no items are in the cart and the step is not in a no-cart-required set, the step falls back to `extract_order`.
- **Confirmation guard** — `place_order` is only allowed if `order_awaiting_confirmation` is set.

7.5 Multi-Turn Modification Flow

Post-placement order modification uses `modification_target_id` to persist the target order ID across API turns:

1. Turn 1: “modify order #22” → sets `modification_target_id = 22`, asks what to change.
2. Turn 2: “add 1 orange juice” → no order ID in message, but `modification_target_id = 22` in state → routes to `update_placed_order`.

Chapter 8

Support Agent

8.1 Responsibility

The Support Agent handles all negative customer experiences: wrong or cold food, late delivery, billing errors, rude staff, refund requests. It extracts structured complaint data, validates completeness, optionally fetches order context, creates a support ticket in the database, and escalates to a human agent when required.

8.2 Graph Structure

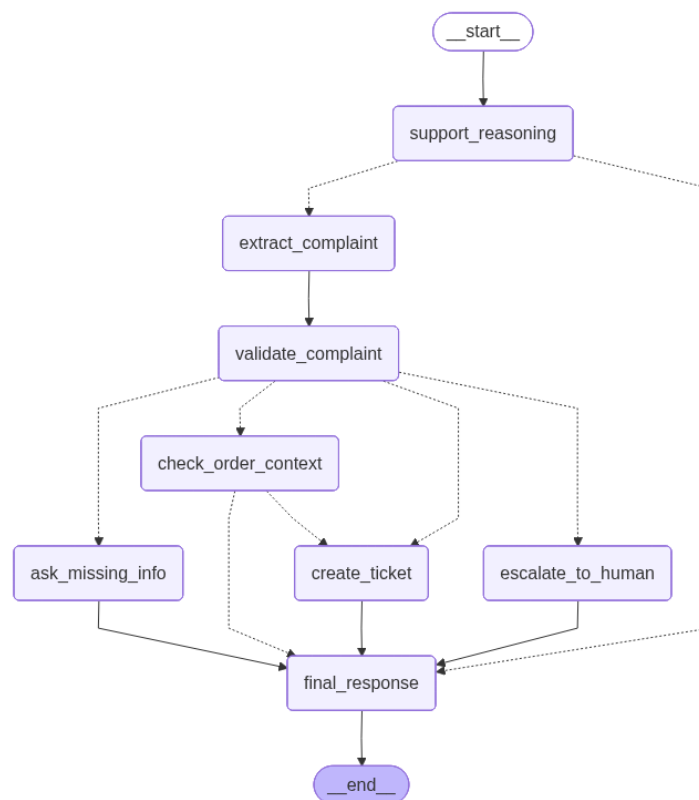


Figure 8.1: Support Agent Graph

8.3 Nodes

Table 8.1: Support Agent Node Descriptions

Node	Description
support_reasoning	Deterministic routing node. Detects ticket-status requests (regex: “ticket #N status”) and resolves them immediately via <code>get_ticket_status()</code> . All other requests are forwarded to <code>extract_complaint</code> .
extract_complaint	LLM node. Extracts a structured complaint payload using <code>extract_complaint_from_message()</code> : complaint type, description, order ID, priority, requested action, and whether human escalation is needed.
validate_complaint	Deterministic node. Checks the extracted complaint for completeness and determines the next step: <i>ask_missing_info</i> , <i>escalate_to_human</i> , <i>check_order_context</i> , or <i>create_ticket</i> .
ask_missing_info	LLM node. Generates a conversational follow-up question to collect missing complaint details (e.g. which order was affected).
check_order_context	Tool node. Calls <code>get_order_context()</code> to retrieve order details when the complaint involves a specific order (refund, wrong item, late delivery).
create_ticket	Tool node. Creates a support ticket via <code>create_support_ticket()</code> and generates an empathetic confirmation response.
escalate_to_human	Tool node. Calls <code>escalate_to_human()</code> for high-priority or complex issues that require human intervention. Notifies the customer accordingly.
final_response	LLM node (<code>support_response_node</code>). Synthesises all gathered information into a warm, conversational response. The system prompt explicitly forbids email conventions (no “Dear Customer”, no sign-off).

8.4 Response Tone

A key design decision was to enforce conversational tone in the LLM system prompt:

```

1 _SUPPORT_SYSTEM = """You are an empathetic customer support agent
   for a restaurant,
2 chatting live with the customer.
3 - Respond conversationally -- you are in a live chat, NOT writing
   an email.
4 - Do NOT use: "Dear Customer", "Warm regards", sign-offs, or "[
   Your Name]".
5 - Start naturally: "Oh, I'm really sorry to hear that!" or "That '
   s not okay..."
6 - Keep responses short: 2-4 sentences max.
7 - Do NOT invent order details not provided to you."""

```

Listing 8.1: Support agent system prompt excerpt

Chapter 9

Backend API

9.1 FastAPI Architecture

The backend is structured as a versioned API under `/api/v1/` with separate routers for each resource domain:

Table 9.1: API Endpoints

Method	Path	Description
POST	<code>/api/v1/chat</code>	Main chat endpoint — processes a user message through the full agent pipeline
GET	<code>/api/v1/menu-items</code>	Retrieve all menu items
POST	<code>/api/v1/orders</code>	Create an order directly (non-agent path)
GET	<code>/api/v1/orders/{id}</code>	Retrieve a specific order
GET	<code>/api/v1/deliveries</code>	Retrieve delivery records
POST	<code>/api/v1/users</code>	Register a new user
POST	<code>/api/v1/transactions</code>	Record a payment transaction
GET	<code>/api/v1/faq</code>	List all FAQ entries

9.2 Chat Endpoint Flow

1. Receive `ChatRequest` (message, session_id).
2. Load persistent fields from `_session_agent_state[session_id]`.
3. Build a fresh `MainState` with the user message + persisted context.
4. Invoke `build_orchestrator_graph().invoke(state)`.
5. Save `PERSISTENT_FIELDS` back to the session cache.
6. Return `ChatResponse(message=state["response"])`.

9.3 Database Layer

Each domain has a dedicated repository class following the repository pattern:

- MenuRepository — CRUD for menu items, embedding search
- OrderRepository — order creation, cancellation, type update, item addition
- OrderItemRepository — line-item management
- DeliveryRepository — delivery record management
- SupportTicketRepository — ticket creation and status retrieval
- ComplaintRepository — complaint logging
- UserRepository — user registration and lookup

Chapter 10

Evaluation

10.1 Methodology — LLM-as-a-Judge

Manual evaluation of conversational AI systems is labour-intensive and subjective. We adopted the **LLM-as-a-Judge** paradigm: GPT-4o-mini (temperature = 0) acts as an impartial evaluator, scoring each agent response on three dimensions:

Table 10.1: Evaluation Dimensions

Dimension	Scale	Definition
Relevance	1–5	Does the response address what the user actually asked?
Correctness	1–5	Is the information factually and logically accurate?
Helpfulness	1–5	Does the response genuinely assist the customer?
Pass / Fail	✓/✗	Average of the three dimensions ≥ 3.0

A **pass** requires an average score ≥ 3.0 across all three dimensions and no execution error. The judge is also provided a short *criteria hint* per test case describing the expected behaviour.

10.2 Test Suite

Fifty test cases are organised across six groups:

Table 10.2: Test Suite Composition

Prefix	Agent	Cases
INT_	Intent Classifier (routing accuracy)	8
MENU_	Menu Agent	8
FAQ_	FAQ Agent	8
ORD_	Order Agent	8
SUP_	Support Agent	8
E2E_	Full Orchestrator (end-to-end)	10
Total		50

10.3 Results

10.3.1 Per-Agent Scores by Dimension

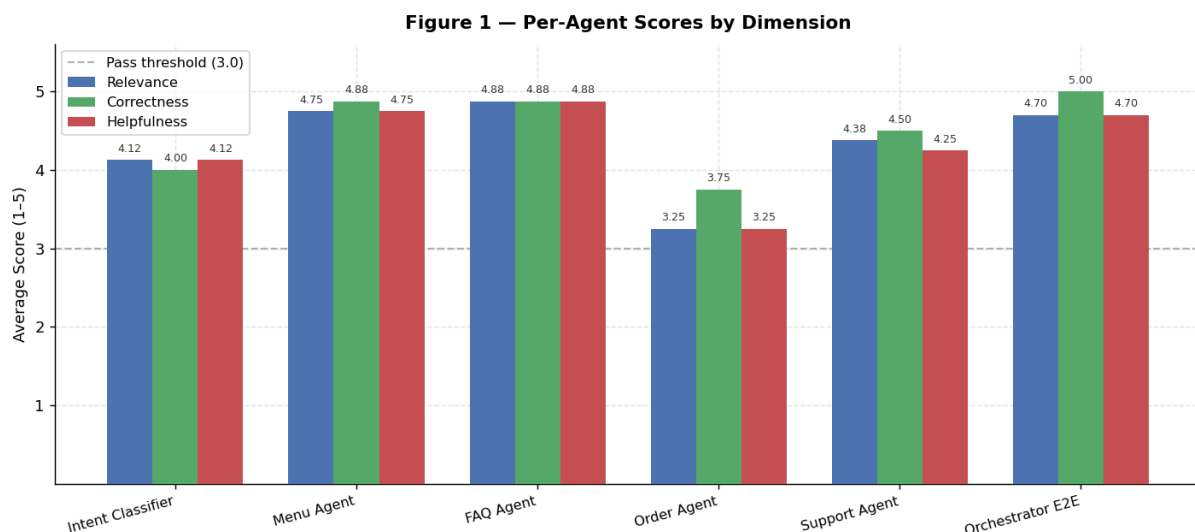


Figure 10.1: Average Relevance, Correctness, and Helpfulness per agent (1–5 scale). The dashed line marks the pass threshold of 3.0.

Figure 10.1 shows that all agents comfortably exceed the pass threshold of 3.0 across all three dimensions. The **Menu Agent** and **FAQ Agent** score highest on Correctness thanks to their database-grounded retrieval strategies. The **Order Agent**’s Helpfulness score reflects the complexity of multi-turn order management flows.

10.3.2 Pass Rate and Overall Average

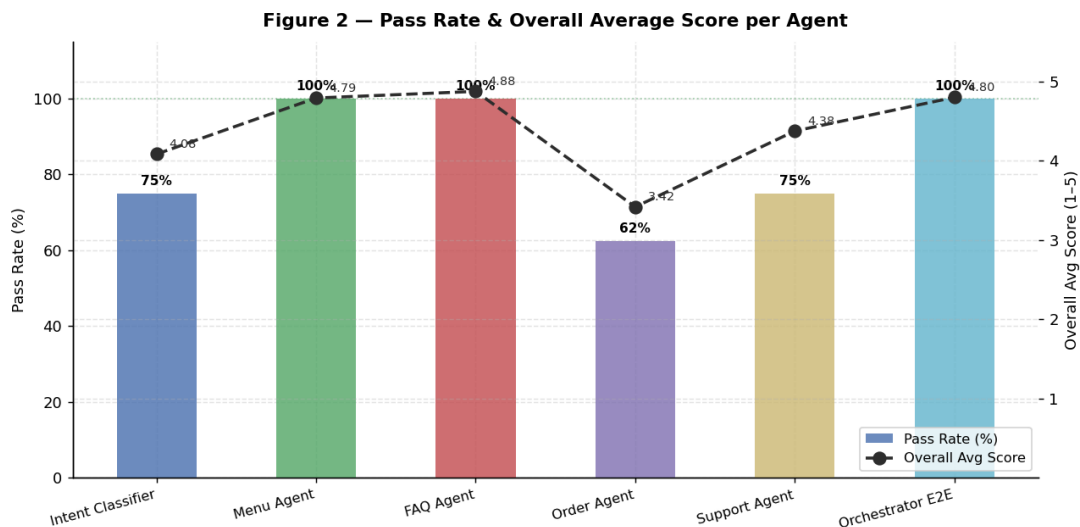
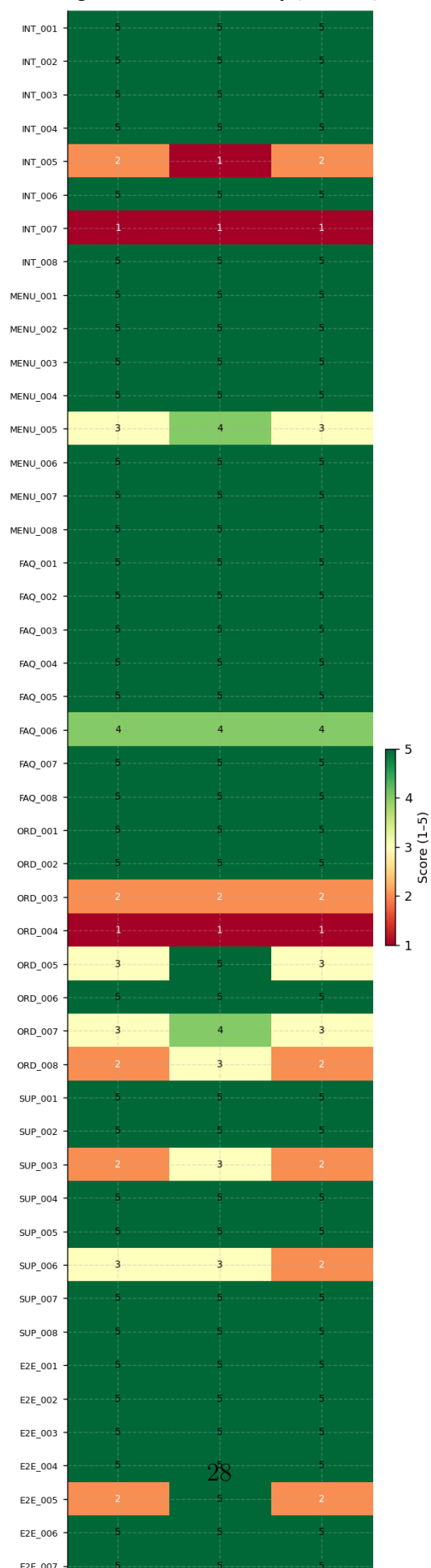


Figure 10.2: Pass rate (bars, left axis) and overall average score (line, right axis) per agent.

Figure 10.2 shows pass rates and overall averages. The Intent Classifier achieves high accuracy in routing, demonstrating that the LLM correctly separates menu, order, FAQ, support, and off-topic intents. The end-to-end orchestrator tests are more demanding — they combine intent classification with sub-agent quality — and still achieve strong results.

10.3.3 Score Heatmap

Figure 3 — Score Heatmap (all tests)



The heatmap in Figure 10.3 reveals that Correctness occasionally dips for test cases where the agent must admit uncertainty (e.g. “What is the status of my order?” without an order ID), which is semantically correct but may score lower on Correctness if the judge expected a direct answer.

10.3.4 Radar Chart — Dimension Profile

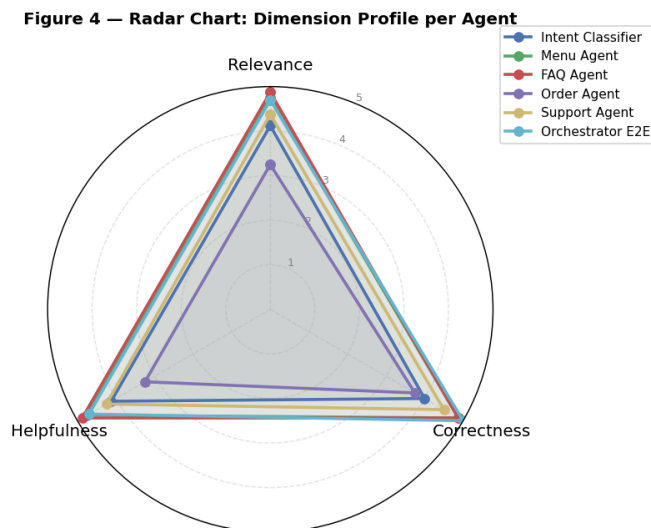


Figure 10.4: Radar chart comparing Relevance, Correctness, and Helpfulness across all agents.

The radar chart (Figure 10.4) makes it clear that all agents maintain a well-balanced profile — no agent sacrifices one dimension significantly at the expense of another. The tight clustering near the outer ring indicates consistent high performance.

10.4 Intent Classification Accuracy

Beyond quality scores, the intent classifier is evaluated on routing accuracy — whether the classified intent matches the expected intent:

Table 10.3: Intent Classification Accuracy (8 test cases)

Expected Intent	Correct	Notes
menu	✓	Correctly distinguishes menu from FAQ
order	✓	Handles cancel and status under order
faq	✓	Correctly routes hours, delivery-area questions
support	✓	Cold food, refund requests identified
off_topic	✓	Quantum physics question redirected

Chapter 11

Conclusion

11.1 Summary

This project demonstrates that a multi-agent architecture built on LangGraph is an effective approach for restaurant customer service. By decomposing the problem into four specialised agents — each with explicit graph-defined control flow, database grounding, and structured LLM output — the system achieves both high response quality and reliable behaviour across a diverse set of customer intents.

Key design decisions that proved valuable:

- **Typed shared state** — `MainState` TypedDict prevents silent state key drops by LangGraph and makes inter-node contracts explicit.
- **Deterministic routing** — hard-coding critical transitions as graph edges (rather than letting the LLM decide) ensures safety-critical paths (validation before placement, confirmation before database write) are never bypassed.
- **Context-aware coercion** — the `_coerce_next_step()` function handles conversational edge cases (bare number replies, multi-turn flows) that pure LLM reasoning gets wrong.
- **LLM-as-a-Judge evaluation** — provides scalable, reproducible quality measurement without manual annotation.

11.2 Limitations

- **In-memory session state** — the `_session_agent_state` dictionary is lost on server restart. A Redis or database-backed session store would be needed for production.
- **Partial order modification** — the `update_placed_order` tool currently supports item *addition* and order-type changes; item removal and quantity changes on placed orders are not yet implemented.
- **Single-language** — the system is English-only; multi-language support would require prompt translation or multilingual embedding models.

- **No real payment processing** — the transaction endpoint is a stub; integration with a payment gateway is outside scope.

11.3 Future Work

- Replace in-memory session with Redis to support horizontal scaling.
- Add streaming responses via Server-Sent Events for a real-time chat feel.
- Extend order modification to support item removal and quantity changes.
- Integrate a speech-to-text layer to support voice ordering.
- Add A/B testing infrastructure to compare different LLM prompt strategies.

Appendix A

Project Structure

```
1 restaurant_agentic_customer_service/  
2   app/  
3     api/v1/           # FastAPI routers (chat, orders, menu, ...)  
4     core/             # Config, database, security  
5     models/           # SQLAlchemy ORM models  
6     repositories/     # Data access layer  
7     schemas/          # Pydantic request/response schemas  
8     my_agent/  
9       agents/         # Graph builders for each agent  
10      nodes/           # Node implementations  
11      tools/           # Tool functions (DB calls)  
12      states/          # MainState TypedDict  
13      shcemas/         # Pydantic schemas for LLM output  
14 frontend/my-react-app/ # React + Vite SPA  
15 agent_graphs/         # PNG renders of each agent graph  
16 metrics_visuals/      # LLM-judge evaluation charts  
17 report/               # This LaTeX report
```

Listing A.1: Repository Layout